TIA — Enterprise Change Impact Intelligence

Product Requirements Document (PRD)

Product	TIA — Technical Impact Analysis
Version	3.1
Target market	Banks and financial-services engineering organisations
Product category	Enterprise software change intelligence
Primary integration	GitHub Enterprise
Delivery model	Multi-tenant SaaS operated by the TIA vendor, plus a dedicated deployment option (vendor-hosted single-tenant or customer-hosted) built from the same code
Document status	Enterprise product specification — every requirement is mandatory for General Availability (GA)
Document set	PRD.md v3.1 · ARCHITECTURE.md v1.3 · TECHNICAL-FRAMEWORK.md v1.3
Last updated	2026-10-28

> **Scope statement.** This is the specification for the complete enterprise product, not an MVP or pilot. All functional (FR), platform (PL), and acceptance (AC) requirements are mandatory for GA. Priority tags only say *which release* delivers a requirement (section 13).
>
> **Precedence.** This PRD owns scope, priorities, releases, and acceptance criteria. ARCHITECTURE.md owns component boundaries and data flow. TECHNICAL-FRAMEWORK.md owns the canonical data model (entity and relationship names, status enums) and engineering standards. Where the documents disagree, the owning document wins and the other must be corrected.

1. Executive Summary

TIA is an evidence-driven change-impact intelligence platform that determines how a proposed software change may affect an enterprise application landscape.

Modern banks operate interconnected systems spanning payment processing, customer onboarding, lending, fraud detection, reconciliation, reporting, APIs, event streams, databases, and deployment environments. These systems are often distributed across many repositories and teams.

Existing tools provide important capabilities: code search, code review, static analysis, dependency scanning, CI/CD, test execution, and runtime observability. TIA does not attempt to replace these tools.

Instead, TIA correlates their evidence into a unified, version-aware technical dependency graph and uses that graph to produce a change-impact report.

When an engineer changes a payment API, for example, TIA should identify known downstream consumers, affected schemas, related tests, deployed services, ownership information, and unresolved dependencies. It should explain why each component may be affected and identify which checks are needed before release.

The product must distinguish between:

- A dependency confirmed by source code or a contract.
- A dependency observed in runtime telemetry.
- A possible dependency inferred from incomplete evidence.
- A dependency that cannot currently be determined.

TIA must never present an unverified relationship as a confirmed fact.

Product vision

Make every consequential software change explainable across code, services, contracts, tests, deployments, and operational history.

Core value proposition

Before a change is merged or released, TIA provides an evidence-backed answer to five questions:

1. What could this change affect?
2. Why does TIA believe each component is affected?
3. Which tests and contract checks should be considered?
4. What is unknown or insufficiently verified?
5. What evidence and approvals are required by the organisation's release policy?

Core deliverable

The core deliverable is not a graph visualisation, an API inventory page, or an AI-generated chat answer.

The core deliverable is an explainable **Change Impact Report** (structured + human-readable) for a proposed revision, with evidence paths, unknowns, and policy outcomes — delivered where the engineer already works (the pull request) and attachable to the bank's change record.

---

2. Problem Statement

2.1 The enterprise engineering problem

A change in one repository can affect systems maintained by different teams, deployed independently, and documented in separate tools.

The relevant information may be spread across:

- GitHub repositories, branches, commits, and pull requests.
- Source code and function-level call relationships.
- REST APIs, asynchronous events, and message schemas.
- Database tables, views, stored procedures, and migrations.
- Unit, integration, contract, and end-to-end tests.

- CI/CD results, build artifacts, and deployment records.
- Runtime traces and service-to-service communication.
- Incidents, rollbacks, and previous changes.
- Service ownership, criticality, and release policies.
- Change records and approvals in IT service-management tools.

An engineer can inspect an individual repository or use a static-analysis tool, but determining the enterprise-wide impact often requires manually correlating information across multiple systems.

2.2 Example

A developer changes the `paymentStatus` field in the `PaymentStatus` response schema of a payment service's REST API.

The potential downstream impact includes:

- A mobile banking backend that calls the API through a generated client.
- A reconciliation service that calls the API directly over HTTP.
- A fraud-monitoring service observed calling the API in production telemetry.
- A notification service and reporting pipeline that consume a `PaymentStatusChanged` event carrying the same field.
- A database mapping or transformation.
- Contract tests and integration tests.
- Services deployed in production that use the changed contract.

The challenge is not simply finding references to `paymentStatus`. It is determining which references belong to which systems, which versions are deployed, which relationships are supported by evidence, and which checks matter for this particular change.

> **Scope note.** Every consumer type in this example is in scope for GA. While a source is not yet connected for a customer, TIA must list it as an *unknown* (e.g. "event consumers not analysed") rather than omit it.

2.3 Why another code graph is insufficient

A graph of files, classes, and functions is useful but is not, by itself, a compelling enterprise product.

Customers already have (or can adopt) code search, AI coding assistants, repository knowledge graphs, MCP tool wiring, static analysis, service catalogs, and observability. A product that only "maps code for chat" will be compared unfavourably to those stacks.

TIA must create value beyond a repository-level graph by correlating:

1. **Code evidence** — what the source appears to do.
2. **Contract evidence** — how systems communicate.
3. **Execution evidence** — what has been observed running.
4. **Test evidence** — what has been tested and with what result.
5. **Deployment evidence** — what versions are running and where.
6. **Operational evidence** — what has failed historically.
7. **Governance evidence** — which policies apply to the change.

The differentiator is the quality of the combined change-impact report, not the existence of the graph itself.

2.4 Lessons from prior product failure (binding constraints)

A prior document-centric "knowledge fabric" demo was rejected. Those failure modes are forbidden in TIA:

Failure	Binding rule
Answers only from documents	Impact authority is technical evidence (code, contracts, tests, runtime, deploy)
Invented API named from query text ("all api")	Census and findings come from extracted/confirmed entities only
No function → chain → consumer path	Every material finding needs an evidence path
Force API catalog upload when none exists	Discover APIs/contracts from code and registries; catalogs are optional enrichers
Impression of dumping entire monorepos onto a permanent god-graph	Incremental, revision-aware, bounded surfaces; report-first UX
LLM used to invent missing dependencies	Deterministic traversal and evidence rules; LLM may only summarise established findings

---

3. Product Goals

3.1 Primary goals

G1. Cross-repository change-impact analysis. Determine potential downstream impact across repositories and services, not just within the changed file or repository.

G2. Evidence-backed explanations. Every material impact finding must include an explanation, supporting evidence, source reference, and evidence status.

G3. Static and runtime correlation. Combine source-derived relationships with available runtime telemetry without treating either source as complete.

G4. Regression planning. Identify relevant existing tests and required verification areas using dependency relationships, contract changes, test results, and available coverage evidence.

G5. Version-aware operational context. Correlate changes with builds, deployments, environments, ownership, and historical incidents when the necessary data is available.

G6. Enterprise release-policy integration. Allow banks to apply configurable release checks based on service criticality, contract compatibility, mandatory tests, unresolved impact, and required approvals — and to attach the resulting evidence to their existing change-management record.

G7. Honest uncertainty. Explicitly identify missing repositories, incomplete telemetry, unresolved consumers, unavailable test evidence, and unsupported analysis.

G8. Low noise. Keep the report short enough to be read during code review. A report that engineers learn to ignore has failed regardless of its recall.

3.2 Secondary goals

- Reduce manual investigation during change reviews.
- Improve consistency of impact assessments between teams.
- Make technical dependencies understandable to engineering leads and release managers.
- Preserve a traceable record of why a change was classified as low, medium, or high impact, suitable as change-control audit evidence.
- Support restricted-network and enterprise deployment models.
- Allow coding assistants (via MCP or API) to retrieve Impact Reports — never to author confirmed dependencies.

3.3 Non-goals

TIA is not intended to:

- Replace GitHub code review or pull-request workflows.
- Replace CodeQL, security scanners, or dependency vulnerability scanners.
- Replace CI/CD or execute tests itself in the initial release.
- Replace contract-testing tools (e.g. Pact); TIA consumes their results as evidence.
- Replace IT service-management or change-management tools (e.g. ServiceNow); TIA supplies evidence to them.
- Replace AI coding assistants (Claude, Copilot, Cursor) as the developer UI.
- Replace general-purpose code knowledge graphs used for assistant context.
- Guarantee that every possible dependency will be discovered.
- Claim that a change is safe solely because no impact was found.
- Automatically approve or deploy high-risk banking changes.
- Use an LLM to invent missing dependencies or unsupported evidence.
- Build a universal business ontology before validating the technical product.
- Require an LLM for deterministic graph traversal or basic impact computation.
- Present a full-estate graph dump as the primary user experience.

---

4. Target Customers and Users

4.1 Initial customer segment

Banks and financial-services organisations with:

- Multiple repositories and independently deployed services.
- Distributed engineering ownership.
- Significant API and event-driven integration.
- Existing CI/CD and code-analysis tooling.
- Formal change-management or release-governance processes.
- A need to explain technical impact for consequential changes.

The first deployment should focus on one bounded application landscape rather than attempting to map an entire bank immediately.

4.2 Landscape onboarding prerequisites

These are prerequisites for onboarding a customer landscape. They are not limits on the product.

Prerequisite	Requirement
Source control	Repositories hosted on a supported platform: GitHub Enterprise (Cloud/Server), GitLab, Bitbucket, or Azure DevOps Repos (FR-001, FR-039).
Languages	Service code in supported languages (FR-004, FR-040); unsupported code is reported as such, never as analysed.
Identity	The customer identity provider supports OIDC or SAML (PL-001).
Service map	Owners can maintain `tia-services.yaml` or an equivalent registry is connected (FR-014 R4).
Deployment	The customer has chosen a deployment model and data locations (section 11.2).
Ground truth	A curated known-dependency set exists or is created during onboarding, for accuracy measurement (section 15).

4.3 Primary personas

Software engineer. Needs to understand which systems, APIs, and tests could be affected by a proposed change.

Senior engineer or technical lead. Needs to review cross-service impact, identify missing verification, and assess whether the proposed implementation is adequately tested.

Engineering manager. Needs to understand blast radius, ownership, and coordination requirements across teams.

Release manager / change manager. Needs evidence that the required checks and approvals have been completed before release, in a form that can be attached to the change record reviewed by the change advisory board (CAB).

Technology risk and compliance. Needs a reproducible, auditable record of how the impact of a change was assessed, to support change-control obligations. *This persona is frequently the budget holder in banks and must sign off customer acceptance.*

Platform engineering team. Needs to integrate repository, CI/CD, deployment, and observability data into a consistent technical model.

Architecture or application owner. Needs a traceable view of dependencies between services, contracts, data stores, and deployed components.

---

5. Core User Journeys

5.1 Pull-request impact analysis

1. An engineer opens or updates a pull request.
2. TIA obtains the base and proposed-head revisions.
3. TIA identifies changed files and changed symbols where supported.
4. TIA determines which APIs, contracts, data entities, and downstream components may be affected.
5. TIA correlates the impact with available tests, runtime evidence, deployment information, and ownership.
6. TIA produces an impact report with evidence and unresolved questions.
7. TIA posts an advisory summary comment and a status check on the pull request.
8. The engineer or reviewer resolves missing evidence or completes the required checks.

5.2 API contract change

1. A developer changes an API definition or event schema.

2. TIA compares the base and proposed contract.
3. TIA identifies breaking or potentially breaking changes using configured compatibility rules.
4. TIA resolves known consumers through code, contract registries, or runtime evidence.
5. TIA identifies consumer-specific tests and verification requirements.
6. TIA reports affected consumers and any consumer relationships that remain unresolved.

5.3 Production incident investigation

1. An operator selects an incident, deployment, or failure signature.
2. TIA resolves the affected runtime component and deployed version when evidence exists.
3. TIA traces known relationships to relevant code, contracts, data stores, and changes.
4. TIA presents candidate changes and dependencies for investigation.
5. Each candidate is labelled according to its evidence.

TIA must not state that a change caused an incident solely because it occurred before the incident.

5.4 Release-readiness assessment

1. A release candidate is associated with a commit, build, and target environment.
2. TIA calculates the impact of the included changes.
3. It evaluates the configured release policy.
4. It checks the availability and results of required tests and contract checks.
5. It reports satisfied, failed, and unevaluated policy requirements.
6. It attaches the report (or a link and summary) to the associated change record, where integration is configured.
7. The authorised release process decides whether to proceed.

---

6. Product Scope and Requirements

Priority definitions. Every requirement is mandatory for GA. The tag states the delivery release only:

- **P0:** Release 1 — Platform foundation and cross-repository impact.
- **P1:** Releases 2–3 — Enterprise evidence, governance, and operational intelligence.
- **P2:** Release 4 — Extended coverage and advanced intelligence.

GA = Releases 1–4 complete, all acceptance criteria (section 14) passing, and the release quality gates (section 19) met.

6.1 Repository and change ingestion

FR-001 — Repository connection (P0)

TIA shall connect to explicitly authorised GitHub repositories.

Requirements:

- Read repository metadata and source code.
- Identify default branches and relevant revisions.
- Retrieve pull-request metadata and changed files where authorised.
- Support private repositories without requiring public exposure.
- Record which repositories were successfully scanned.

- Report permission failures and inaccessible repositories.

FR-002 — Revision-aware analysis (P0)

TIA shall distinguish the base revision from the proposed revision.

A pull-request analysis must use the actual revision identifiers rather than silently analysing the current default branch.

The report must identify the commit or revision used for each analysis.

FR-003 — Incremental analysis (P0)

TIA should reanalyse changed components and the relationships potentially affected by those changes instead of rescanning every repository for every pull request.

A full rescan must remain available for recovery, parser upgrades, and graph reconciliation.

FR-004 — Supported languages (P0)

TIA shall analyse, at GA, at least:

- **Java** — Spring Boot / Spring Web (routes), `RestTemplate`, `WebClient`, OpenFeign (outbound calls), JPA entities, Flyway/Liquibase migrations.
- **Python** — FastAPI / Flask / Django REST (routes), `requests` / `httpx` (outbound calls), SQLAlchemy / Django ORM models, Alembic / Django migrations.

Both are P0. Further languages are delivered through the adapter SDK (FR-040).

The architecture must support adding additional languages through separate language-analysis adapters.

The product must clearly report language and framework coverage. Unsupported code must not be represented as fully analysed.

FR-005 — Repository freshness (P0)

TIA shall track:

- Last successful scan.
- Source revision analysed.
- Parser or extractor version.
- Scan failures.
- Stale relationships.
- Missing or inaccessible repositories.

A partial scan must not be presented as a complete scan.

6.2 Technical dependency graph

FR-006 — Core graph entities (P0)

The initial graph shall support the following entities:

- Repository.
- Commit or revision.

- File.
- Function or method.
- API endpoint.
- Service.
- Test case (unit or integration).
- API schema.
- Data entity and database table, where declared through a supported ORM or migration (FR-006a).
- Service owner, where available.

FR-006a — Data-access scope. Data relationships from declarative sources (ORM entity/model mappings and versioned migration files) are P0. Raw/dynamic SQL and stored procedures are covered by FR-041 (P2); both are mandatory for GA. Where a service is known to use undeclared data access, the report must say "data access not fully analysed" rather than imply there is none.

FR-007 — Execution and operations entities (P1)

The graph shall support:

- Event and event schema.
- Build artifact.
- Deployment.
- Environment.
- Runtime trace or observed dependency.
- Incident.
- Failure signature.
- Rollback.
- Service criticality.
- Release policy.
- Change record (external ITSM reference).

FR-008 — Explicit relationship types (P0)

Relationships shall be typed rather than represented as generic links. Each type must have exactly one defined meaning.

The canonical relationship vocabulary — names, direction, and meaning — is defined in TECHNICAL-FRAMEWORK.md §5.2 and is binding on all three documents and on the implementation. It includes, among others: `CALLS`, `HANDLES`, `EXPOSES`, `CONSUMES_API`, `PRODUCES_EVENT`, `CONSUMES_EVENT`, `USES_SCHEMA`, `READS`, `WRITES`, `MIGRATES`, `TESTS`, `COVERS`, `BUILT_FROM`, `DEPLOYED_AS`, `OBSERVED_CALL_TO`, `OWNED_BY`, `GOVERNED_BY`, `ASSOCIATED_WITH_INCIDENT`.

In particular, synchronous API consumption (`CONSUMES_API`) and asynchronous event consumption (`CONSUMES_EVENT`) are distinct relationship types.

FR-009 — Relationship provenance (P0)

Every material relationship must preserve:

- Source system.
- Source file, symbol, or record.
- Revision or time range, where applicable.
- Extraction method and, for resolved cross-repository relationships, the resolution rule identifier (FR-014).
- Evidence reference.

- Evidence status (FR-010).
- Last-seen or validity information.

FR-010 — Evidence classifications (P0)

TIA shall distinguish at least these states. The machine-readable value is shown in `code`; the same values are used in the graph, the API, and the report.

Classification	Value	Meaning
Source-confirmed	`source_confirmed`	Supported by an explicit source-code or configuration relationship
Contract-confirmed	`contract_confirmed`	Supported by an API, event, or schema definition
Runtime-observed	`runtime_observed`	Observed in available runtime telemetry
Corroborated	`corroborated`	Supported by two or more independent evidence sources of different kinds
Inferred	`inferred`	Produced by a named, deterministic inference rule from indirect evidence; requires validation
Unknown	`unknown`	Insufficient evidence to determine the relationship
Stale	`stale`	Evidence may no longer describe the current revision or deployment

A source-confirmed relationship is not necessarily proof that the path executes in production. A runtime-observed relationship is evidence of observed execution, not proof of all possible execution paths.

Trust rule: Only classifications allowed by the active analysis/policy configuration may appear as confirmed impact. Inferred relationships must be labelled and must not be silently upgraded.

Inference rule: An `inferred` relationship may only be produced by an enumerated, versioned rule listed in TECHNICAL-FRAMEWORK.md §5.4. Each inferred relationship records the rule identifier. Neither an LLM nor free-text name similarity may produce an inferred relationship.

FR-011 — Temporal relationships (P1)

TIA shall preserve historical versions of material relationships when their meaning depends on a commit, build, deployment, or environment.

The graph must be able to answer questions such as:

- Which consumers were known to use this contract at the time of a release?
- Which version of a service was deployed when an incident occurred?
- Which dependencies changed between two releases?

6.3 Cross-repository and cross-service impact

FR-012 — Changed-symbol identification (P0)

TIA shall identify changed files and, where supported, changed functions, methods, classes, and configuration elements.

If symbol-level comparison is unavailable, the report shall state that the analysis is file-level rather than silently claiming symbol-level precision.

FR-013 — Dependency traversal (P0)

TIA shall traverse the relevant graph from changed components to potential downstream dependants.

The traversal must support both direct and transitive dependencies.

For example:

```
Changed Function → API Endpoint → Consumer Service → Consumer Test
```

The report must preserve the path connecting each affected component to the original change.

FR-014 — Cross-repository resolution (P0)

TIA shall connect relationships across repository boundaries only through the enumerated resolution strategies below. Each resolved relationship records which strategy produced it.

ID	Strategy	Example	Resulting status
R1	Generated or shared client	Consumer imports a client library generated from, or published by, the provider	`source_confirmed`
R2	Fully resolved HTTP call site	Consumer call with a literal or constant path template; base URL comes from a configuration key that maps to the provider via a service registry, gateway route table, or the TIA service map	`source_confirmed`
R3	Declared contract	Consumer-side OpenAPI reference or consumer-driven contract (e.g. Pact) naming the provider endpoint	`contract_confirmed`
R4	Service map	Owner-maintained `tia-services.yaml` in source control mapping configuration keys / hostnames to services	Contributes to R2; does not by itself create a consumer edge
R5	Path-template match with unresolved host	Consumer path template matches exactly one provider endpoint in scope, but the base URL cannot be resolved	`inferred` (rule `INF-PATH-UNIQUE`)
R6	Runtime observation (P1)	Trace shows calls from consumer to provider endpoint	`runtime_observed`

Requirements:

- Matching names alone (service names, field names, class names) must never be sufficient to create a relationship of any status.
- A call site that cannot be resolved by R1–R5 must be listed in the report's unknowns as an **unresolved outbound call**, with its source location.
- Each analysis must report its **resolution rate**: resolved outbound call sites ÷ all detected outbound call sites in the analysed repositories, broken down by strategy.
- The resolution rate is a release acceptance metric (section 15.3).

FR-015 — Impact scope (P0)

TIA shall classify findings into:

- Direct impact.
- Transitive impact.
- Contract impact.

- Data impact.
- Test impact.
- Runtime or deployment impact.
- Operational or historical association.
- Unresolved potential impact.

FR-016 — Bounded traversal (P0)

The system shall enforce configurable traversal depth, result limits, and analysis time budgets.

When a limit is reached, the report must indicate that the result is truncated or incomplete.

TIA must not silently omit potentially relevant findings and label the analysis complete.

6.4 API and event contract intelligence

FR-017 — API discovery (P0)

TIA shall discover supported API definitions from source code and configuration, including framework-supported route declarations for every supported language and framework (FR-004, FR-040).

For each endpoint, record where available:

- HTTP method.
- Path.
- Service and repository.
- Request and response schema.
- Authentication or authorisation configuration.
- Source revision.
- Related tests.

No catalog gate: Absence of an uploaded OpenAPI/Swagger document must not prevent discovery of endpoints declared in supported frameworks. External catalogs, if connected, enrich and corroborate; they do not define the only path to an API inventory.

FR-018 — API contract comparison (P0 basic / P1 full)

TIA shall compare base and proposed API definitions and identify changes.

P0 (basic, REST only): removed endpoint; changed HTTP method or path; removed response field; field type change; request field becoming required.

P1 (full): enum narrowing; response shape change; authentication or authorisation configuration change; configurable compatibility rules per protocol and schema format.

TIA must not claim that a contract change is backward compatible without evaluating the applicable compatibility rules. In P0, changes outside the basic list are reported as "compatibility not evaluated".

FR-019 — Event and message contracts (P1)

TIA shall support configured event or message schemas, including applicable formats such as JSON Schema and Avro, and topic-level producer/consumer declarations (e.g. Kafka listener and producer configuration).

It shall identify known producers, consumers, and schema dependencies when evidence is available.

FR-020 — Consumer impact report (P0 for REST / P1 for events)

For each changed contract, TIA shall report:

- Changed contract elements.
- Known affected consumers.
- Evidence for each consumer relationship.
- Compatibility classification.
- Relevant tests.
- Unresolved consumers or missing contract information.

6.5 Test intelligence

FR-021 — Static test association (P0)

TIA shall link tests to functions, endpoints, contracts, or services when a defensible relationship can be established from imports, test structure, explicit configuration, or other supported evidence.

The association method must be recorded. Associations based solely on naming conventions must be labelled `inferred`.

FR-022 — CI test results (P1)

TIA shall ingest authorised CI results, including:

- Test identity.
- Test result.
- Execution time.
- Commit or build identifier.
- Environment, where available.
- Test report or evidence reference.

FR-023 — Coverage enrichment (P1)

Where available, TIA shall ingest coverage or execution evidence linking tests to source symbols.

TIA must distinguish:

- Test exists.
- Test is statically associated with a component.
- Test passed in a particular run.
- Test executed a particular component.
- Test validated a particular contract.

These states are not interchangeable.

FR-024 — Regression recommendations (P1)

TIA shall recommend tests based on affected components and available test relationships.

Recommendations must explain why a test is relevant.

TIA must not claim that unselected tests can safely be skipped solely because no graph path was found.

FR-025 — Missing-test evidence (P1)

TIA shall identify potentially important impact areas for which no relevant test evidence was found.

The report must say "no relevant test evidence found in the analysed sources," not "no tests exist," unless the available test inventory is known to be complete.

6.6 Runtime and deployment intelligence

FR-026 — Runtime integration (P1)

TIA shall support ingestion of authorised runtime dependency evidence from supported observability sources, such as OpenTelemetry-compatible traces.

The initial implementation may use imported or queried telemetry rather than requiring TIA to collect traces directly. Telemetry is ingested into the evidence store before analysis; an analysis never queries telemetry live (see TECHNICAL-FRAMEWORK.md §6).

FR-027 — Runtime relationship classification (P1)

TIA shall label runtime relationships with the observed service, environment, time window, and available deployment or version identifiers.

Absence of a runtime observation must not be interpreted as proof that a dependency does not exist.

FR-028 — Deployment correlation (P1)

TIA shall link services to build artifacts and deployments where reliable identifiers are available.

The system must distinguish the proposed code revision from the revision currently deployed in each environment.

FR-029 — Environment-specific impact (P1)

TIA shall support queries that distinguish development, testing, staging, and production environments.

If environment mapping is incomplete, the report must explicitly identify the limitation.

6.7 Historical change and incident intelligence

FR-030 — Change history (P1)

TIA shall retain authorised historical information about changes and relevant graph relationships.

It should support comparisons between releases and identification of dependencies introduced, removed, or modified over time.

FR-031 — Incident association (P1)

TIA shall support linking incidents to affected services, deployments, time windows, and related changes when evidence is available.

Each association must identify whether it is:

- Directly recorded.
- Time-correlated.
- Dependency-derived.
- Manually validated.

FR-032 — Historical investigation (P1)

TIA shall allow users to investigate which changes and components are relevant to a historical incident.

It must not automatically classify a historically associated change as the root cause.

6.8 Release-policy and workflow integration

FR-033 — Configurable policy rules (P1)

Authorised administrators shall be able to configure release requirements based on criteria such as:

- Service criticality.
- Breaking API or event changes.
- Mandatory test results.
- Required contract checks.
- Missing owner approval.
- Unresolved high-consequence dependencies.
- Stale or incomplete impact analysis.
- Required evidence for production deployment.

FR-034 — Policy evaluation (P1)

Each policy rule shall produce one of the following outcomes: `pass`, `fail`, `not_evaluated`, `requires_review`.

A missing data source must not automatically produce a pass.

FR-035 — GitHub pull-request integration (P0 comment / P1 check)

P0 — advisory comment. TIA shall post (and update in place on new pushes) a single pull-request comment containing:

- Analysis status and revisions analysed.
- Impact summary (counts by severity and evidence status).
- The material findings (section 7.4), each with a one-line reason.
- Unknowns that affect confidence.
- Report link or identifier.
- Incomplete-analysis warning, when applicable.

The P0 comment never blocks merging.

P1 — status check. TIA shall publish a GitHub check run carrying the same content plus required verification and policy outcomes.

FR-036 — Advisory and enforcement modes (P1)

TIA shall support:

1. **Advisory mode** — report findings without blocking merges (default).
2. **Policy mode** — report policy outcomes to the configured CI/CD or repository protection process.

The customer must explicitly configure enforcement. TIA shall not independently deploy code or override repository protection settings.

FR-037 — Assistant / MCP distribution (P1)

TIA may expose read APIs or an MCP server so coding assistants can fetch an existing Impact Report by analysis id / PR id.

Requirements:

- Tools return structured report JSON.
- Assistants may narrate only fields present in the report.
- Assistants must not be able to write confirmed relationships through MCP in the initial design.
- Org guidance: "no blast-radius claim in review without a TIA analysis id."

FR-038 — Change-management integration (P1)

TIA shall be able to attach an Impact Report summary and permanent link to a change record in a supported IT service-management tool (ServiceNow first), keyed by a change-record identifier supplied in the PR, release metadata, or API call.

Requirements:

- Write only to the configured change record; never create, approve, or transition change records.
- Record the analysis id, revisions, status, policy outcomes, and unknowns in the attachment.
- If the analysis is not `complete`, the attachment must say so in its first line.

6.9 Extended coverage and advanced intelligence

FR-039 — Additional source-control platforms (P2)

TIA shall support GitLab (self-managed and SaaS), Bitbucket (Data Center and Cloud), and Azure DevOps Repos with the same capabilities as GitHub: repository connection, revision-aware analysis, merge/pull-request reporting (comment and status), and permission-aware access.

FR-040 — Language adapter SDK and additional languages (P2)

TIA shall provide a documented adapter interface so new languages can be added without changing the engine, and shall ship adapters for at least C#/.NET (ASP.NET Core, HttpClient, Entity Framework), TypeScript/JavaScript (Express/NestJS, fetch/axios), Go (net/http, chi/gin), and Kotlin (Spring).

FR-041 — SQL and stored-procedure data lineage (P2)

TIA shall extract READS/WRITES relationships from embedded SQL strings, SQL files, and stored-procedure definitions in supported dialects (PostgreSQL, Oracle, SQL Server, DB2), and link stored procedures to the services that call them. Dynamic SQL that cannot be parsed is reported as an unknown.

FR-042 — Event-stream lineage (P2)

TIA shall trace events across multiple hops (producer → topic → consumer → re-published topic) and into data pipelines declared in source control (e.g. Kafka Streams, Spark job definitions), using the same evidence rules.

FR-043 — Change-risk trends (P2)

TIA shall report descriptive trends per application and team: change volume, share of breaking changes, unresolved-dependency rate, missing-verification rate, and incident associations over time. Trends must not be presented as probabilities.

FR-044 — Severity-rule calibration from validated outcomes (P2)

TIA shall record reviewer verdicts on findings (confirmed / false positive / irrelevant) and incident outcomes, and report per-rule precision so administrators can tune severity and inference rules. Rule changes are versioned and never rewrite historical reports.

FR-045 — Multi-team change coordination (P2)

For material findings, TIA shall notify the owning team of each affected component (chat, email, or ITSM task, as configured), track acknowledgement, and show coordination status in the report and policy evaluation.

6.10 Platform requirements

ID	Requirement	Release
PL-001	Single sign-on via OIDC and SAML 2.0; service accounts with scoped API tokens	P0
PL-002	Role-based access control (viewer, engineer, release manager, auditor, administrator) and per-user repository permission mirroring from the SCM	P0
PL-003	Multi-tenant isolation: shared schema with `tenant_id` on every tenant-owned row, enforced by PostgreSQL row-level security; per-tenant configuration, connectors, users, and audit; no cross-tenant query path	P0
PL-004	Horizontal scaling of workers and API; no single point of failure in the supported HA topology	P1
PL-005	Backup, restore, and disaster recovery with documented RPO/RTO	P1
PL-006	Administration console and configuration API (connectors, service map, policies, rules, retention)	P1
PL-007	Platform observability: metrics, structured logs, traces, health endpoints (TECHNICAL-FRAMEWORK.md §14)	P0
PL-008	Versioned public REST API and OpenAPI specification; client SDK	P1
PL-009	Container images and Helm chart for customer-controlled Kubernetes; Docker Compose for single-node installs	P0
PL-010	Air-gapped installation (no outbound network required)	P1
PL-011	Encryption in transit and at rest; secrets in an approved secrets manager (Vault, cloud KMS)	P0
PL-012	Accessibility (WCAG 2.1 AA) for the web UI	P1

ID	Requirement	Release
PL-013	Tenant lifecycle: self-service and assisted onboarding, SCM app installation per tenant, suspension, offboarding with verified data deletion	P0
PL-014	Usage metering per tenant (repositories, analyses, users) and plan limits; billing-system export	P1
PL-015	Data residency: tenant pinned to a hosting region (at least EU, US, and one APAC region); data never leaves the region	P1
PL-016	Per-tenant encryption keys, including customer-managed keys (BYOK via cloud KMS)	P1
PL-017	Per-tenant rate limiting and fair scheduling so one tenant cannot starve others	P0
PL-018	SaaS operational compliance: SOC 2 Type II and ISO 27001 control set, vulnerability management, penetration testing, incident response runbooks	P1

---

7. Impact Analysis Algorithm

The impact engine is the central technical capability of TIA.

7.1 Inputs

The engine receives:

- Base revision.
- Proposed revision.
- Changed files and symbols.
- Relevant graph subgraph, pinned to an evidence snapshot.
- Contract differences.
- Available test evidence.
- Runtime and deployment evidence.
- Applicable policy rules.
- Analysis limits and configuration.

All inputs are read from TIA's own stores. The engine does not call external systems during an analysis; this is what makes results reproducible.

7.2 Processing stages

Stage 1 — Change detection. Identify changed components and classify the change type: implementation change; API change; event-schema change; database-schema change; configuration change; test-only change; dependency or build change.

Stage 2 — Graph resolution. Resolve changed components to known graph entities and determine which relationships can be traversed. Unresolved symbols, ambiguous service identities, and inaccessible repositories must be recorded.

Stage 3 — Impact traversal. Traverse relevant dependency relationships to identify direct and transitive impact. Traversal rules shall be relationship-aware. For example, a write dependency may have different implications from a test

association or an incident association.

Stage 4 — Contract and data analysis. Compare changed contracts or schemas against their previous versions and identify known consumers or data dependencies.

Stage 5 — Test and runtime correlation. Associate affected components with relevant tests, prior CI outcomes, coverage evidence, observed runtime relationships, and deployed versions.

Stage 6 — Evidence assessment. For each finding, record the evidence path, evidence status, source freshness, and unresolved limitations.

Stage 7 — Severity and materiality. Apply the severity rules (7.3) and select material findings (7.4).

Stage 8 — Policy evaluation. Evaluate applicable release rules using the available evidence.

Stage 9 — Report generation. Produce a structured report and a human-readable summary.

7.3 Impact severity

TIA shall use configurable severity categories. Severity is computed by documented, versioned rules from **two inputs**: the *nature of the change* on the path (breaking vs non-breaking, as determined by FR-018) and the *criticality* of the affected component. Criticality alone never produces Critical.

Severity	Definition	Default rule (configurable)
Critical	A change may break a critical system or violates a mandatory compatibility or release policy	Breaking contract change **and** affected consumer is tagged critical; **or** a mandatory policy evaluates to `fail`
High	A consequential downstream dependency or important contract is affected	Breaking contract change with any confirmed consumer; **or** non-breaking change reaching a critical consumer via a confirmed path
Medium	A relevant downstream component or verification requirement is identified	Non-breaking contract or implementation change with a confirmed downstream consumer; **or** any breaking change reaching a consumer only via an `inferred` path
Low	Limited, localised impact is supported by the available evidence	Impact confined to the changed repository, or test-only / internal change
Unknown	The evidence is insufficient to assess the impact reliably	Change type could not be classified, or the only paths are `unknown`

Additional rules:

- A finding reached **only** through `inferred` relationships is capped at Medium, unless the configured policy explicitly overrides the cap.
- When no criticality data is configured, all services are treated as non-critical and the report states "criticality not configured".
- Severity must not be presented as a statistically calibrated probability unless a validated model supports that claim.

7.4 Materiality and noise budget

A **material finding** is any finding with severity Critical, High, or Medium, or any finding on a path containing a breaking contract change.

The PR comment and the report summary show material findings first. Non-material findings are collapsed but available. Report noise is measured continuously (section 15) and the default thresholds may be tuned per customer.

---

8. Change Impact Report

Every analysis shall produce a structured, machine-readable report and a readable summary.

8.1 Required report fields

- Analysis identifier.
- Repository and pull-request identifiers, where applicable.
- Change-record identifier, where supplied.
- Base and proposed revisions.
- Evidence snapshot identifier and scan timestamp.
- Analysis status (8.3).
- Coverage summary, including the cross-repository resolution rate (FR-014).
- Changed components.
- Affected components.
- Severity and rationale (including the severity rule applied).
- Evidence paths.
- Relevant tests and their evidence status.
- Contract findings.
- Runtime and deployment findings.
- Unresolved dependencies and unresolved outbound calls.
- Missing data sources.
- Truncation or time-budget warnings.
- Policy results.
- Recommended next checks.

8.2 Example output

The following is illustrative output, not a claim about a real repository.

```
{
  "analysis_id": "tia-pr-1842",
  "status": "complete_with_gaps",
  "change": {
    "repository": "payments-service",
    "pull_request": 1842,
    "base_revision": "base-sha",
    "proposed_revision": "head-sha",
    "changed_components": [
      {
        "type": "api_schema",
        "name": "PaymentStatus",
        "change": "field_type_changed",
        "breaking": true
      }
    ]
```

```
  },
  "coverage": {
    "repositories_analysed": 4,
    "repositories_inaccessible": 0,
    "outbound_call_sites": 37,
    "outbound_call_sites_resolved": 31,
    "resolution_by_strategy": { "R1": 12, "R2": 15, "R3": 2, "R5": 2 }
  },
  "findings": [
    {
      "component": "reconciliation-service",
      "impact_type": "contract",
      "severity": "high",
      "severity_rule": "SEV-BREAKING-CONFIRMED",
      "evidence_status": "source_confirmed",
      "resolution_strategy": "R2",
      "reason": "reconciliation-service calls GET /payments/{id}/status, whose PaymentStatus response field
changed type.",
      "path": [
        "payments-service:PaymentStatus",
        "payments-service:GET /payments/{id}/status",
        "reconciliation-service:StatusClient.fetch"
      ],
      "required_verification": [
        "Run the reconciliation contract tests",
        "Verify compatibility with the deployed consumer version"
      ]
    },
    {
      "component": "ledger-sync",
      "impact_type": "contract",
      "severity": "medium",
      "severity_rule": "SEV-INFERRED-CAP",
      "evidence_status": "inferred",
      "resolution_strategy": "R5",
      "inference_rule": "INF-PATH-UNIQUE",
      "reason": "ledger-sync calls a path matching GET /payments/{id}/status, but its base URL could not be
resolved.",
      "limitations": [
        "Base URL comes from an environment variable not present in source"
      ]
    }
  ],
  "unknowns": [
    {
      "area": "unresolved_outbound_calls",
      "count": 6,
      "reason": "6 outbound HTTP call sites could not be resolved to a provider."
    },
    {
      "area": "event_consumers",
      "reason": "Event consumers are not analysed in this release."
    },
    {
      "area": "external_consumers",
      "reason": "The external consumer inventory was not available."
```

```
    }
  ],
  "policy_results": [
    {
      "rule": "required_contract_tests",
      "result": "not_evaluated",
      "reason": "The CI test result was unavailable."
    }
  ]
}
```

8.3 Report integrity and analysis status

The analysis `status` takes exactly one of the values below (canonical definition in TECHNICAL-FRAMEWORK.md §7):

Status	Meaning
`running`	Analysis in progress; no conclusions may be shown
`complete`	All in-scope sources analysed; no limits reached; no unknowns that affect material findings
`complete_with_gaps`	Analysis ran to completion, but unknowns, unresolved call sites, or missing optional sources exist
`partial`	A limit was reached, data was stale, or one or more in-scope repositories/sources failed
`failed`	Analysis could not produce findings (e.g. changed component unresolvable, base/head unavailable)

The report must distinguish a successful analysis with no identified impacts from: a scan that did not run; a failed scan; a partial scan; an analysis performed against stale data; a result truncated by resource limits; an analysis that could not resolve the changed component.

A clean result is meaningful only when the scope and limitations of the analysis are visible.

8.4 Anti-hallucination integrity

The following behaviours are defects, not UX quirks:

- Creating an entity whose name is derived from query phrasing (e.g. an API named "all api").
- Promoting an inferred relationship to source-confirmed without new evidence.
- Producing an inferred relationship without a recorded inference rule.
- Omitting the unknowns panel when consumer inventory or scan coverage is incomplete.
- Claiming "safe to skip tests" solely because no path was found.
- Treating "no runtime observation" as "no dependency."

---

9. User Interface Requirements

9.1 Pull-request impact view

The primary interfaces — the PR comment and the web report — shall show:

1. Overall impact classification and analysis status.
2. Changed components.

3. Most consequential affected services (material findings).
4. Contract and data changes.
5. Relevant tests and their results.
6. Policy outcomes.
7. Unknowns and missing evidence.

Users must be able to navigate from a finding to its supporting source.

9.2 Impact path explorer (P1)

Users shall be able to inspect a dependency path such as:

```
Changed API → Event Producer → Event Schema → Reconciliation Consumer → Contract Test
```

The interface must identify the evidence supporting each edge and distinguish confirmed relationships from inferred relationships.

A large graph visualisation must not be the only way to understand the result. The default experience should prioritise the impact report, with graph exploration available as supporting functionality.

9.3 Filters (P1)

The report shall support filtering by: severity; service or repository; direct versus transitive impact; contract, data, test, runtime, and operational impact; evidence status; environment; policy result; unresolved or stale evidence.

9.4 Unknowns panel (P0)

The report shall display missing information that materially affects confidence in the assessment. Examples:

- Consumer repository not connected.
- Outbound call sites not resolved (with locations).
- Contract registry unavailable.
- Event consumers not analysed.
- No relevant coverage data found.
- Runtime telemetry does not cover the target period.
- Deployment version could not be resolved.
- Analysis reached its configured traversal limit.

---

10. System Architecture

Component design is owned by ARCHITECTURE.md. This section states only the product-level constraints the architecture must satisfy.

10.1 Logical layers

Layer	Responsibility
A — Source integrations	Connectors for GitHub Enterprise, source and configuration, API and event-schema registries, CI/CD test reports, coverage, OpenTelemetry-compatible telemetry, deployment systems, incident management, ITSM change records, and ownership/policy registries. Each connector operates with explicit permissions and reports its coverage.
B — Evidence extraction	Language and source adapters extract symbols and calls, API endpoints and outbound call sites, contract definitions, data-access relationships, test associations, and build/deployment metadata, retaining provenance. Optional adapters may import graphs from other AST/knowledge-graph tools when provenance can be preserved; import does not transfer authority.
C — Identity and relationship resolution	Resolve identities across repositories and systems using only the strategies in FR-014. Ambiguous matches remain unresolved or inferred until corroborated.
D — Technical dependency graph	Store entities, typed relationships, evidence, and temporal validity. Support bounded traversal, revision-aware analysis, and targeted retrieval.
E — Change-impact engine	Compare revisions, traverse, assess contracts, associate tests, correlate runtime evidence, and compute severity — reading only from TIA's stores.
F — Policy, report, and integrations	Evaluate policies; produce structured reports, UI, GitHub PR results, ITSM attachments, and optional assistant/MCP read access.

10.2 Architecture principle

The graph is the evidence and relationship layer. The impact engine determines what to do with that evidence. The report communicates conclusions and limitations.

Do not confuse graph construction with impact analysis.

10.3 Storage

Storage is selected for the first deployment's scale and operational requirements. For an early local prototype, SQLite can be sufficient. For a multi-repository enterprise deployment, the design must evaluate concurrent access, incremental updates, historical revisions, traversal performance, auditability, backup and recovery, retention, and operational support.

The PRD does not mandate a particular graph database. Storage selection must be validated against the expected workload.

10.4 LLM usage

The first version must not depend on an LLM to determine graph reachability or invent dependency relationships.

An LLM may optionally summarise already-established findings, but:

- Structured findings must be generated by deterministic logic.
- Each material claim must refer to evidence.
- Unsupported claims must be excluded.
- Summaries must preserve unknowns and qualifications.
- The report must remain useful if an LLM is unavailable.
- In restricted deployments, LLM use is disabled unless the customer explicitly approves the model endpoint.

---

11. Security, Privacy, and Enterprise Deployment

Security is a product requirement, not a later enhancement. **Every item in 11.1, 11.3, and 11.5 is P0**, because no bank deployment can start without them.

11.1 Access control (P0)

- Use least-privilege access to repositories and integrations.
- Do not request write access when read access is sufficient.
- Separate read permissions from the ability to publish pull-request comments/checks.
- Respect repository and organisation access boundaries.
- Do not expose information from repositories the requesting user is not authorised to access — including in cross-repository findings shown on another repository's pull request (show "1 finding in a repository you cannot access" instead).

11.2 Deployment models

TIA is delivered as **multi-tenant SaaS** operated by the TIA vendor. The **same code base** must also run as a dedicated deployment for customers that cannot use shared infrastructure.

Model	Who operates	Database	Tenancy	Release
Multi-tenant SaaS	TIA vendor	Vendor-operated PostgreSQL, shared schema	Many tenants, isolated by PostgreSQL row-level security (PL-003)	P0
Dedicated SaaS	TIA vendor	Separate vendor-operated database per customer	One tenant (same code, tenancy enforced identically)	P1
Customer-hosted (private / on-premises)	Customer	Customer PostgreSQL	One tenant, or business units as tenants	P1
Air-gapped customer-hosted	Customer	Customer PostgreSQL	As above, no outbound network (PL-010)	P1

Requirements common to all models:

- Tenant isolation is enforced in the database (row-level security), not only in application code; a tenant id is bound to every connection/session.
- Source code is fetched transiently for extraction; the platform stores only extracted entities, relationships, and evidence references (file, line, revision), never full repository copies, unless the tenant explicitly enables source snippets.
- Every deployment documents exactly where source code, graph data, reports, and telemetry are processed and stored, including the SaaS hosting region(s) (PL-015).
- Developer-local mode exists for evaluation and testing only (non-production code).

11.3 Audit (P0)

Record: repository access and scan events; source revisions analysed; connector failures; analysis requests; report generation; policy evaluations; administrative configuration changes; evidence updates and invalidations; external writes (PR comments, checks, ITSM attachments).

11.4 Data retention (P1)

Customers must be able to configure retention for: source-derived graph records; historical analysis reports; runtime metadata; incident associations; audit records.

Secrets, credentials, and unnecessary personal data must not be copied into graph records or reports (P0).

11.5 Integrity (P0)

TIA shall preserve the source, revision, and evidence snapshot behind each finding so that an auditor or engineer can reproduce or investigate the result.

Changes to extraction logic must not silently rewrite historical reports without identifying the new analysis version.

---

12. Performance and Reliability Requirements

The following are release acceptance targets. They are measured on the reference landscape (section 15) and confirmed against each customer's SLAs during onboarding. Values marked *proposed* must be agreed with the first customer.

Requirement	Target
Incremental PR analysis	P95 ≤ 2 minutes, landscape of up to 500 repositories with the graph already built
PR comment / status latency	P95 ≤ 3 minutes from push
Initial landscape scan	≤ 4 hours for 500 repositories / 20 M lines of code (*proposed*)
Interactive report	P95 ≤ 5 seconds after the analysis is complete
Concurrent analyses	≥ 50 concurrent PR analyses per deployment (*proposed*)
Availability	99.9% monthly for the HA topology (*proposed*)
Recovery	RPO ≤ 1 hour, RTO ≤ 4 hours (*proposed*)
Analysis determinism	Same inputs, extractor version, configuration, and evidence snapshot produce the same structured findings
Resource limits	Configurable traversal, memory, and time limits
Failure reporting	Connector and analysis failures are visible, not silently ignored
Incremental consistency	Changes invalidate or refresh affected relationships
Reproducibility	Every report records the revisions, evidence snapshot, and analysis configuration used

Performance benchmarks must use representative repositories and documented test fixtures.

---

13. Enterprise Delivery Plan

TIA is delivered as one enterprise product in four releases. Every release is production quality: secure, tested, documented, and deployable. Nothing in this plan is a pilot, prototype, or optional "evaluate later" item. GA is reached when Release 4 is complete.

This release plan is canonical. ARCHITECTURE.md §9 and TECHNICAL-FRAMEWORK.md §15 refer to it and must not define a different sequence.

Release 1 — Platform foundation and cross-repository impact (P0)

- Platform: multi-tenant PostgreSQL storage with row-level security, tenant lifecycle and per-tenant fair scheduling, persisted versioned graph, job queue, idempotent APIs, evidence snapshots (FR-003, FR-005, PL-003, PL-013, PL-017).
- Security: SSO (OIDC/SAML), RBAC, SCM permission mirroring, audit, secrets, encryption (PL-001, PL-002, PL-011, section 11 P0).
- GitHub App connector: registration, webhooks, revision fetch, PR comment (FR-001, FR-002, FR-035 P0).
- Java and Python adapters incl. ORM/migration data access (FR-004, FR-006, FR-006a).
- Cross-repository resolution R1–R5, traversal, basic contract diff, severity, report, unknowns (FR-008–FR-018 P0 scope, FR-020 REST, FR-021).
- Stale-relationship handling (AC-011).
- Deployment: SaaS production environment (one region), container images, Helm chart, Compose; platform observability (PL-007, PL-009).
- Accuracy harness and reference landscape (section 15).

Release 2 — Enterprise evidence and governance (P1)

- CI test results, coverage, regression recommendations, missing-test evidence (FR-022–FR-025).
- Full contract comparison; event and message contracts (FR-018 full, FR-019, FR-020 events).
- Runtime telemetry (R6), deployment correlation, environment views (FR-026–FR-029).
- Usage metering and plan limits; data residency (additional regions); per-tenant keys / BYOK; dedicated SaaS model (PL-014, PL-015, PL-016).
- Ownership and criticality registry; release-policy engine; GitHub check run; advisory and enforcement modes (FR-033–FR-036).
- ITSM change-record integration (FR-038); MCP / read API (FR-037).
- Web UI: report, path explorer, filters, unknowns panel (section 9); administration console (PL-006); accessibility (PL-012).

Release 3 — Operational intelligence and scale (P1)

- Temporal relationships, change history, release comparisons (FR-011, FR-030).
- Incident association and historical investigation (FR-031, FR-032).
- Retention enforcement and audit reporting (section 11.4).
- HA, horizontal scale, backup/DR, customer-hosted and air-gapped install, public API + SDK, SOC 2 / ISO 27001 readiness (PL-004, PL-005, PL-008, PL-010, PL-018).

Release 4 — Extended coverage and advanced intelligence (P2) → GA

- GitLab, Bitbucket, Azure DevOps (FR-039).
- Adapter SDK and C#/.NET, TypeScript/JavaScript, Go, Kotlin adapters (FR-040).
- SQL / stored-procedure lineage; event-stream lineage (FR-041, FR-042).
- Change-risk trends, calibration from validated outcomes, multi-team coordination (FR-043–FR-045).

Definition of done (every release)

- All acceptance criteria assigned to the release pass in automated tests.
- Release quality gates (section 19) met.
- Security review, dependency scan, and threat-model update completed.
- Upgrade and migration path from the previous release tested.
- Administrator, user, and API documentation updated.

---

14. Acceptance Criteria

The product is not complete merely because it can build a graph or list API endpoints. The "Release" column states which release must satisfy each criterion; all must pass for GA.

ID	Criterion	Release
AC-001 Changed revision	Given a pull request with a known base and proposed revision, TIA reports the exact revisions analysed.	1
AC-002 Direct impact	Given a changed function with a known caller, TIA identifies the caller and provides a source-backed path.	1
AC-003 Transitive impact	Given a chain of supported dependencies across two or more repositories, TIA identifies the reachable downstream components within the configured analysis scope.	1
AC-004 Contract impact	Given a changed API contract and a known consumer, TIA identifies the consumer and explains the relationship using contract or source evidence.	1
AC-005 Unresolved consumer	Given a changed contract and an incomplete consumer inventory, TIA reports the known consumers and explicitly states that the inventory may be incomplete.	1
AC-006 Test association	Given a test with an established relationship to an affected component, TIA includes the test and explains the association method.	1
AC-007 Runtime distinction	Given a runtime-observed dependency, TIA labels it as observed and identifies the relevant time window and environment when available.	2
AC-008 No false coverage claim	Given no runtime evidence for a dependency, TIA does not conclude that the dependency cannot occur.	2
AC-009 Failed scan	Given an inaccessible repository or failed parser, TIA reports the failure and marks any resulting analysis as `partial` or `failed`.	1
AC-010 Zero-impact result	Given a successful analysis with no identified downstream impact, TIA returns a successful, scoped result with scan coverage and limitations. It does not confuse this result with a failed or empty scan.	1
AC-011 Stale relationships	Given a relationship invalidated by a source change, TIA does not continue presenting it as current without identifying its stale status.	1
AC-012 Bounded traversal	Given a graph that exceeds configured analysis limits, TIA reports truncation and does not claim complete impact discovery.	1
AC-013 Policy failure	Given a mandatory contract check that failed, TIA reports the failed requirement and the supporting result.	2
AC-014 Policy not evaluated	Given an unavailable CI result, TIA marks the relevant policy as `not_evaluated` rather than `pass`.	2

ID	Criterion	Release
AC-015 Reproducible evidence	Given a report, an authorised reviewer can identify the source revisions, evidence snapshot, and evidence used to produce its material findings, and re-running the analysis on the same snapshot yields the same findings.	1
AC-016 Access control	Given a user without permission to a repository, TIA does not disclose that repository's protected source or findings, including on another repository's PR comment.	1
AC-017 No invented entities	Given a request to list APIs or consumers when none were extracted, TIA returns an empty confirmed set with an explicit incompleteness or empty-extract status. It never invents entity names from the request phrasing.	1
AC-018 Resolution transparency	Given outbound call sites that cannot be resolved, TIA lists each one with its source location and reports the overall resolution rate.	1
AC-019 Inference labelling	Given a relationship produced by an inference rule, TIA labels it `inferred`, names the rule, and does not rate the finding above Medium unless policy overrides the cap.	1
AC-020 PR comment	Given a pushed revision on an open PR in an analysed repository, TIA posts or updates exactly one advisory comment reflecting the latest analysed revision and never blocks the merge.	1
AC-021 Incremental update	Given a push that changes one file, TIA re-extracts only affected evidence and the resulting graph equals a full rescan.	1
AC-022 Freshness	Given a repository whose last scan failed, the report states the last successful scan and analysed revision.	1
AC-023 Java analysis	Given Java/Spring services, endpoints, outbound calls (RestTemplate/WebClient/Feign), and JPA data access are extracted with source locations.	1
AC-024 Data impact	Given a migration that drops a column, services whose ORM mappings read or write that table are reported as data impact.	1
AC-025 SSO and RBAC	Given an authenticated user without the release-manager role, policy configuration is refused; all access is audited.	1
AC-026 GitHub App	Given a webhook for a pushed PR, TIA fetches both revisions through the GitHub App and posts or updates one comment.	1
AC-027 CI evidence	Given CI results, the report distinguishes test exists / associated / passed / executed / validated.	2
AC-028 Event impact	Given a changed event schema with a known consumer, TIA reports the consumer with event evidence.	2
AC-029 Deployed version	Given deployment records, the report distinguishes the proposed revision from the revision deployed in each environment.	2
AC-030 Enforcement mode	Given policy mode, a failed mandatory policy is reported to the configured protection process; advisory mode never blocks.	2
AC-031 ITSM	Given a change-record id, the report summary is attached to that record and nothing else is modified.	2
AC-032 MCP	Given an analysis id, the MCP tool returns the report JSON; no write tool exists.	2
AC-033 Temporal query	Given two releases, TIA lists dependencies introduced, removed, and modified between them.	3
AC-034 Incident	Given an incident, candidate changes are labelled by association type and none is labelled root cause.	3

ID	Criterion	Release
AC-035 Tenant isolation	Given two tenants, no API call, job, or direct SQL session bound to one tenant can read or write the other tenant's rows (enforced by row-level security).	1
AC-036 Additional SCM	Given a GitLab merge request, TIA behaves as for a GitHub pull request (AC-001, AC-020).	4
AC-037 SQL lineage	Given a stored procedure that writes a changed table, its calling services are reported with SQL evidence.	4
AC-038 Calibration	Given reviewer verdicts, per-rule precision is reported; historical reports are unchanged.	4
AC-039 Tenant lifecycle	Given an offboarded tenant, all its rows and stored evidence are deleted and the deletion is recorded in the audit trail.	1
AC-040 Fair scheduling	Given one tenant submitting 1,000 analyses, another tenant's analysis still starts within its configured latency target.	1
AC-041 Data residency	Given a tenant pinned to a region, no tenant data is written to storage outside that region.	2

---

15. Success Metrics

Each release and each customer onboarding must measure whether TIA improves engineering decisions, not just whether it extracts graph entities.

15.1 Ground truth

Recall and precision are measured against a **curated known-dependency set** for the TIA reference landscape (maintained by the TIA team) and for each customer landscape:

- Built **before** TIA results are shown, by a customer-nominated reviewer who is not on the TIA team, from existing architecture documentation, owner interviews, and a manual review of sample changes.
- Augmented afterwards with any TIA-found dependency that the reviewer confirms is real (tracked separately as "missed by existing workflow").
- Frozen per measurement round and versioned.

15.2 Primary product metrics

Impact discovery

- Recall: percentage of curated in-scope dependencies discovered by TIA.
- Number of material dependencies found by TIA that the existing workflow missed.
- Precision: percentage of material findings the reviewer confirms are real.
- Cross-repository resolution rate (FR-014).

Noise

- Median and P90 number of material findings per PR.
- Percentage of PR comments engineers mark as "not useful".

Engineering efficiency

- Time required to complete an impact assessment.
- Time spent manually locating downstream consumers.
- Time required to identify the relevant regression tests.
- Reviewer time spent investigating missing or ambiguous evidence.

Evidence quality

- Percentage of material findings with traceable evidence.
- Percentage of findings with resolvable source references.
- Rate of stale relationships.
- Percentage of analyses correctly identifying incomplete input coverage.

Operational usefulness

- Percentage of recommendations considered useful by engineers.
- Percentage of recommended checks that engineers confirm are relevant.
- Number of cross-team dependencies identified before release.
- Policy evaluation accuracy against independently reviewed cases.

15.3 Release acceptance targets

Every release must meet these targets on the reference landscape before shipping, and on each customer landscape before go-live.

Metric	Target
Recall on curated in-scope critical dependencies	≥ 90%
Precision of material findings	≥ 80%
Material findings per PR	Median ≤ 5, P90 ≤ 15
Cross-repository resolution rate	≥ 70% of outbound call sites
Material findings with traceable evidence	≥ 90% (target 100%)
Reduction in median time to complete the selected impact-assessment workflow	≥ 30%
Silent pass for missing mandatory evidence	0
Material findings presented as `source_confirmed` without supporting evidence	0

Results should be segmented by language, relationship type, resolution strategy, and evidence source.

---

16. Risks and Mitigations

Risk	Mitigation
1 — The product duplicates existing tools	Integrate with existing code analysis, repository, CI/CD, contract-testing, ITSM, and observability tools. Compete on the Impact Report + unknowns + policy — not on "we also have a code graph."
2 — Cross-repository relationships are incomplete	Enumerated resolution strategies (FR-014), resolution-rate reporting, owner-maintained service map, contract and runtime evidence, unresolved call sites listed explicitly, release acceptance threshold.

Risk	Mitigation
3 — Engineers distrust impact results	Show the evidence path for every material finding, expose extraction methods and resolution strategies, separate confirmed, observed, inferred, stale, and unknown relationships.
4 — Reports are too noisy and get ignored	Severity driven by change nature × criticality (7.3); inferred-only cap; materiality filter (7.4); noise metrics and targets (15.3).
5 — The graph becomes too large	Incremental ingestion, targeted graph expansion, bounded traversal, query-specific reports.
6 — Runtime telemetry creates false confidence	Record the observation window and environment. Treat missing observations as unknown, not proof of absence.
7 — Historical correlation is mistaken for causation	Label incident relationships by evidence type and require independent validation before claiming root cause.
13 — Banks reject shared SaaS	Dedicated SaaS and customer-hosted models from the same code (11.2); database-enforced isolation; residency and BYOK; SOC 2 / ISO 27001.
8 — The platform creates a security or compliance concern	Security baseline in Release 1; private deployment; audit logging; data minimisation; configurable retention; customer-controlled enforcement.
9 — Full enterprise scope is large	Four-release plan; each release independently shippable, tested, and documented; strict layer boundaries so evidence sources are added without engine rework.
10 — Language coverage	Java and Python in Release 1; adapter SDK and four more languages in Release 4.
11 — Buyer is not the user	Engineers use the PR comment; risk/change management fund the product. Both sign off customer acceptance; FR-038 links the two.
12 — Ground truth is biased	Curated set built before results are shown, by a reviewer outside the TIA team (15.1).

17. Competitive Positioning

TIA should coexist with existing developer and security tools. Named products are examples of each category, not an exhaustive list.

Capability	Example tools	What they do	TIA's intended role
Code search and navigation	Sourcegraph, GitHub code search	Locate symbols and references	Correlate findings across repositories and systems; resolve references to services and revisions
AI-assisted coding / review	Claude, GitHub Copilot, Cursor	Review proposed code and suggest improvements	Supply evidence-backed Impact Reports those tools can cite, not invent
Repository knowledge graphs / assistant MCP maps	Various	Map code for assistant context	Consume or coexist; own revision-aware cross-system impact + report integrity
Service catalogs / developer portals	Backstage, Port, Cortex	Record services, owners, and declared dependencies	Use as ownership/criticality evidence; verify declared dependencies against code
Static analysis	CodeQL, SonarQube	Detect code defects and patterns	Use relevant results as evidence for impact

Capability	Example tools	What they do	TIA's intended role
Dependency and security scanning	Snyk, Dependabot	Identify package dependencies and known risks	Connect relevant dependency changes to affected applications and releases
Contract testing	Pact / PactFlow	Verify consumer–provider contracts	Use contracts as R3 evidence; show which contracts a change touches and whether they were verified
CI/CD	GitHub Actions, Jenkins	Build, test, and deploy	Interpret test and deployment evidence in the context of change impact; evaluate configured policies
Runtime observability / service maps	Datadog, Dynatrace, OpenTelemetry backends	Show observed behaviour and service interactions	Correlate observed relationships with code, contracts, and proposed changes
Change management	ServiceNow	Record and approve changes	Attach reproducible impact evidence to the change record
TIA	—	—	Produce an explainable, cross-system Change Impact Report for a specific revision and evaluate configured verification policies

Intended differentiation (one paragraph for buyers)

Banks already have search, scanners, CI, observability, service catalogs, and increasingly AI assistants with code graphs. TIA's job is to answer, for a specific proposed revision, what else may be affected across services, why (evidence class + path), what is unknown, and whether release policy is satisfied — and to put that answer in the pull request and the change record, without pretending a chat answer or a repo map is a release decision.

This is an intended differentiation, not a claim that existing platforms cannot provide any overlapping functionality. The product must demonstrate incremental value in a customer environment.

---

18. Customer Onboarding and Validation

Validation is continuous and per customer; it does not gate product scope.

For each customer landscape:

1. Confirm onboarding prerequisites (section 4.2) and deployment model.
2. Connect repositories, identity provider, and evidence sources; build the initial graph.
3. Build or import the curated known-dependency set (section 15.1).
4. Run TIA on at least 10 recent changes, including at least 3 contract changes, and compare with the customer's existing process.
5. Measure recall, precision, noise, resolution rate, and assessment time against section 15.3 targets.
6. Obtain sign-off from engineering and from the change/risk function that the report is acceptable change-control evidence.
7. Go live in advisory mode; enable policy (enforcement) mode when the customer chooses.

Commercial questions (owned outside this PRD): pricing unit (per repository, per engineer, per landscape); buyer (engineering platform vs technology risk); support model for private installs.

19. Release Quality Gates

Each release ships only when all of the following hold. These are quality gates on a release, not decisions about whether to build the product.

Gate	Pass criterion
Functional	All acceptance criteria assigned to the release pass in automated tests
Accuracy	Section 15.3 targets met on the reference landscape
Performance	Section 12 targets met on the reference landscape
Security	Security review passed; no open critical/high vulnerabilities; threat model updated
Operability	Install, upgrade, backup/restore, and observability verified on the supported topologies
Documentation	Administrator, user, and API documentation complete for the release scope

20. Final Product Definition

TIA is an enterprise change-impact intelligence platform that connects code, APIs, event schemas, data dependencies, tests, runtime observations, deployments, and operational history into a version-aware evidence graph.

Its core deliverable is not a graph visualisation, an API inventory, or an AI-generated explanation.

Its core deliverable is an explainable Change Impact Report that identifies potentially affected systems, provides the evidence behind each finding, recommends relevant verification, reports unresolved dependencies, and evaluates configured release policies.

The primary workflow is determining, inside the pull request, the cross-repository impact of a proposed change; the enterprise product also covers contract changes, release readiness, and incident investigation end to end.

The long-term opportunity is to become a trusted intelligence layer between software changes and enterprise release decisions — working with existing engineering tools rather than attempting to replace them.

Product success criterion: A bank's engineering team can use TIA to identify important cross-system dependencies, verify why they matter, and make a better-informed change or release decision than it could using its existing workflow alone.

21. Document history

Version	Date	Notes
0.1	—	Basic code-scan / ask CLI PRD
0.2	—	Competitive rewrite (authority vs assistant graphs)
2.0	2026-10-28	Enterprise Change Impact Intelligence PRD for banks; GitHub-first; report-centric; evidence classifications; phased MVP and gates; binding demo-failure constraints; AC-017; FR-037 MCP read path; Java pilot note; assistant coexistence

Version	Date	Notes
2.1	2026-10-28	Review fixes: document precedence; Gate 0 and pilot qualification criteria; single pilot language (Java default); enumerated cross-repo resolution strategies with resolution-rate metric (FR-014); inference rules; severity defined as change nature × criticality with inferred cap; materiality and noise budget; advisory PR comment promoted to P0; basic REST contract diff to P0; data access limited to ORM/migrations in MVP; event promotion rule; ITSM change-record integration (FR-038) and risk/compliance persona; security baseline moved to P0; canonical status and evidence-status enums; fixed invalid JSON example; precision, noise, and resolution targets; ground-truth method; gate thresholds; named competitors; AC-018–AC-020; removed local file path
3.0	2026-10-29	Enterprise scope: removed MVP/pilot framing; every requirement mandatory for GA; priorities now map to Releases 1–4; both Java and Python required; Phase 4 items made requirements FR-039–FR-045; platform requirements PL-001–PL-012; enterprise performance/availability targets; acceptance criteria AC-021–AC-038; gates replaced by release quality gates; onboarding replaces pilot qualification
3.1	2026-10-29	Delivery model decided: multi-tenant SaaS (shared schema + PostgreSQL row-level security) with dedicated and customer-hosted options from the same code; PL-003 moved to Release 1; new PL-013–PL-018 (tenant lifecycle, metering, residency, per-tenant keys, fair scheduling, SaaS compliance); AC-035 moved to Release 1; AC-039–AC-041; risk 13

End of PRD v3.1